

PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Procesamiento de Lenguaje Natural

github.com/marcelomendoza/IIC3670

Marcelo Mendoza

Interpretabilidad mecanística

La interpretabilidad mecanística estudia cómo calcula un modelo lo que responde. Tres técnicas sobre transformers de NLP, desde leer un estado interno hasta intervenirlo.

Técnica	Pregunta	Evidencia
logit lens	¿Qué predice el modelo en cada capa?	lectura del residual
probing	¿Qué propiedad lingüística se puede leer de una capa?	correlación
activation patching	¿Qué componente transporta la información de la respuesta?	causal

Elhage et al., 2021; Belinkov, Computational Linguistics 2022; Jurafsky y Martin, Speech and Language Processing (borrador 2026).

El residual stream

En un transformer pre-norm, cada atención y cada red por posición leen el estado de la posición y le suman su salida. El estado final es la suma de todas esas contribuciones.

$$h_i^{(0)} = x_i$$

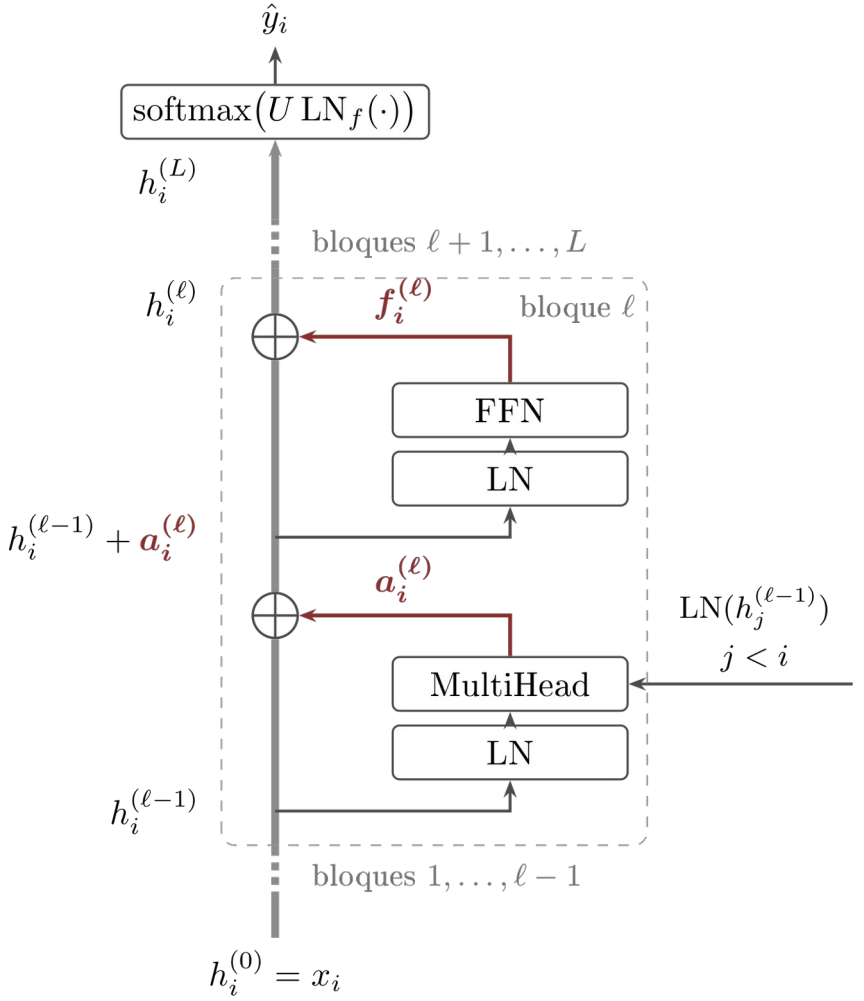
$$\boldsymbol{a_i^{(\ell)}} = \mathrm{MultiHead}\big(\mathrm{LN}(h_i^{(\ell-1)}),\ [\mathrm{LN}(h_j^{(\ell-1)})]_{j\le i}\big)$$

$$\boldsymbol{f_i^{(\ell)}} = \mathrm{FFN}\big(\mathrm{LN}(h_i^{(\ell-1)} + a_i^{(\ell)})\big)$$

$$h_i^{(\ell)} = h_i^{(\ell-1)} + \boldsymbol{a_i^{(\ell)}} + \boldsymbol{f_i^{(\ell)}}$$

$$\hat{y}_i = \mathrm{softmax}\big(U\,\mathrm{LN}_f(h_i^{(L)})\big)$$

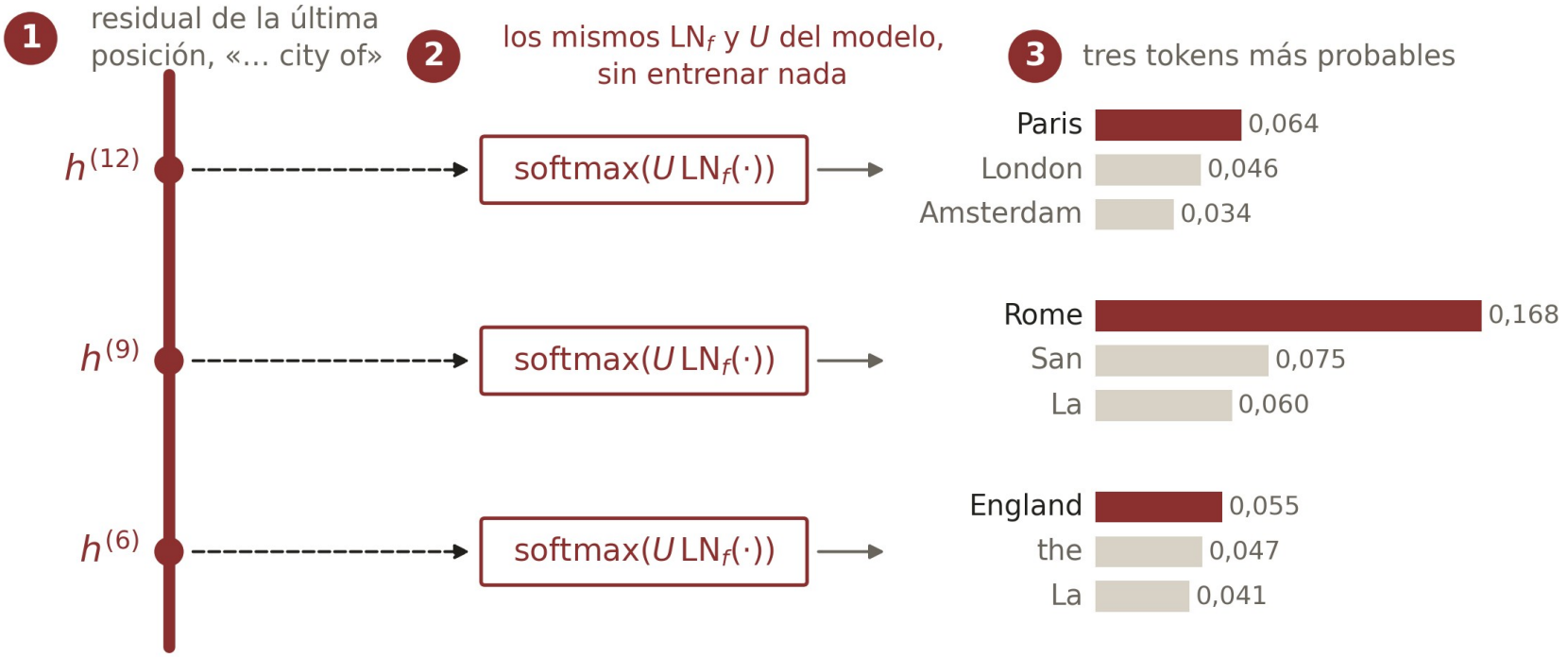
$$h_i^{(L)} = x_i + \sum_{\ell=1}^{L}\big(\boldsymbol{a_i^{(\ell)}} + \boldsymbol{f_i^{(\ell)}}\big)$$



Elhage et al., 2021 (A Mathematical Framework for Transformer Circuits); Jurafsky y Martin (Speech and Language Processing).

Logit lens: el mecanismo

Se corre el modelo una vez y se guarda el residual de cada capa. Cada estado intermedio se lee con la LayerNorm final y el cabezal U del propio modelo, como si el modelo terminara en esa capa.



GPT-2 small, última posición de «The Eiffel Tower is located in the city of». nostalgebraist, LessWrong 2020. Cálculo propio.

Logit lens: la definición

Para cada capa ℓ y posición i, la logit lens define una distribución sobre el vocabulario. De ella se leen el token más probable, la probabilidad de la respuesta y su rango.

$$\hat{y}_i^{(\ell)} = \mathrm{softmax}\big(U\,\mathrm{LN}_f(\boldsymbol{h}_i^{(\ell)})\big) \in \mathbb{R}^V, \qquad \ell = 0, \ldots, L$$

una distribución por capa ℓ y por posición i

$$\hat{y}_i^{(L)} = \hat{y}_i$$

en la última capa coincide con la predicción del modelo

$$\mathrm{top}^{(\ell)} = \arg\max_v \ \hat{y}_i^{(\ell)}[v]$$

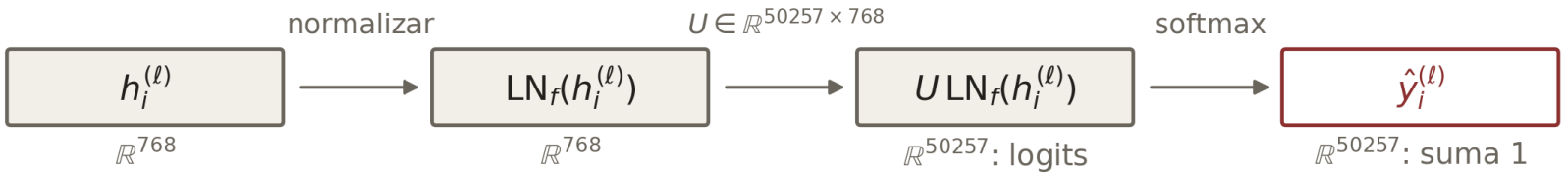
el token más probable en la capa ℓ

$$p^{(\ell)}(w) = \hat{y}_i^{(\ell)}[w]$$

la probabilidad de la respuesta w

$$\mathrm{rango}^{(\ell)}(w) = 1 + \big|\{v : \hat{y}_i^{(\ell)}[v] > \hat{y}_i^{(\ell)}[w]\}\big|$$

1 si w es el más probable; V si es el último



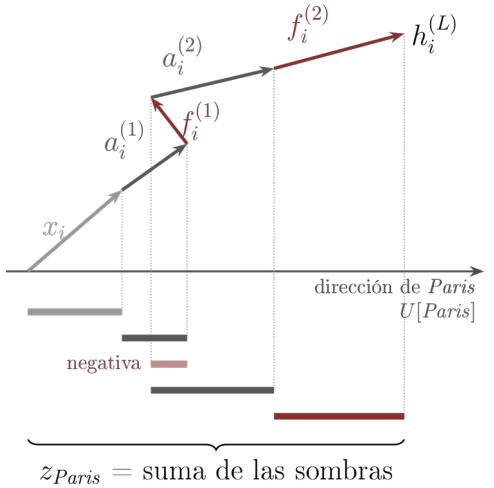
nostalgebraist, LessWrong 2020 (interpreting GPT: the logit lens); Belrose et al., 2023.

Atribución directa al logit

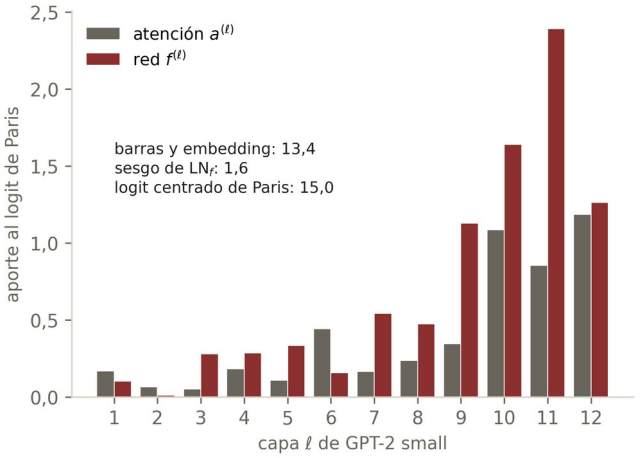
El logit de un token mide cuánto apunta el estado final hacia ese token. Como el estado es una suma, el logit también lo es, con un aporte por subcapa.

$$z_w = U[w]^\top h_i^{(L)} = \underbrace{U[w]^\top \beta}_{\text{sesgo de LN}_f} + \underbrace{U[w]^\top x_i}_{\text{embedding}} + \sum_{\ell=1}^{L} \Big(\underbrace{U[w]^\top a_i^{(\ell)}}_{\text{DLA de la atención } \ell} + \underbrace{U[w]^\top f_i^{(\ell)}}_{\text{DLA de la FFN } \ell} \Big)$$

$U[w]$: fila de U del token w, su dirección en el residual. Cada término pasa por LN_f con su escala fija.



Cada flecha es lo que escribe una subcapa. Su sombra sobre la dirección del token es su DLA, que puede ser negativa.

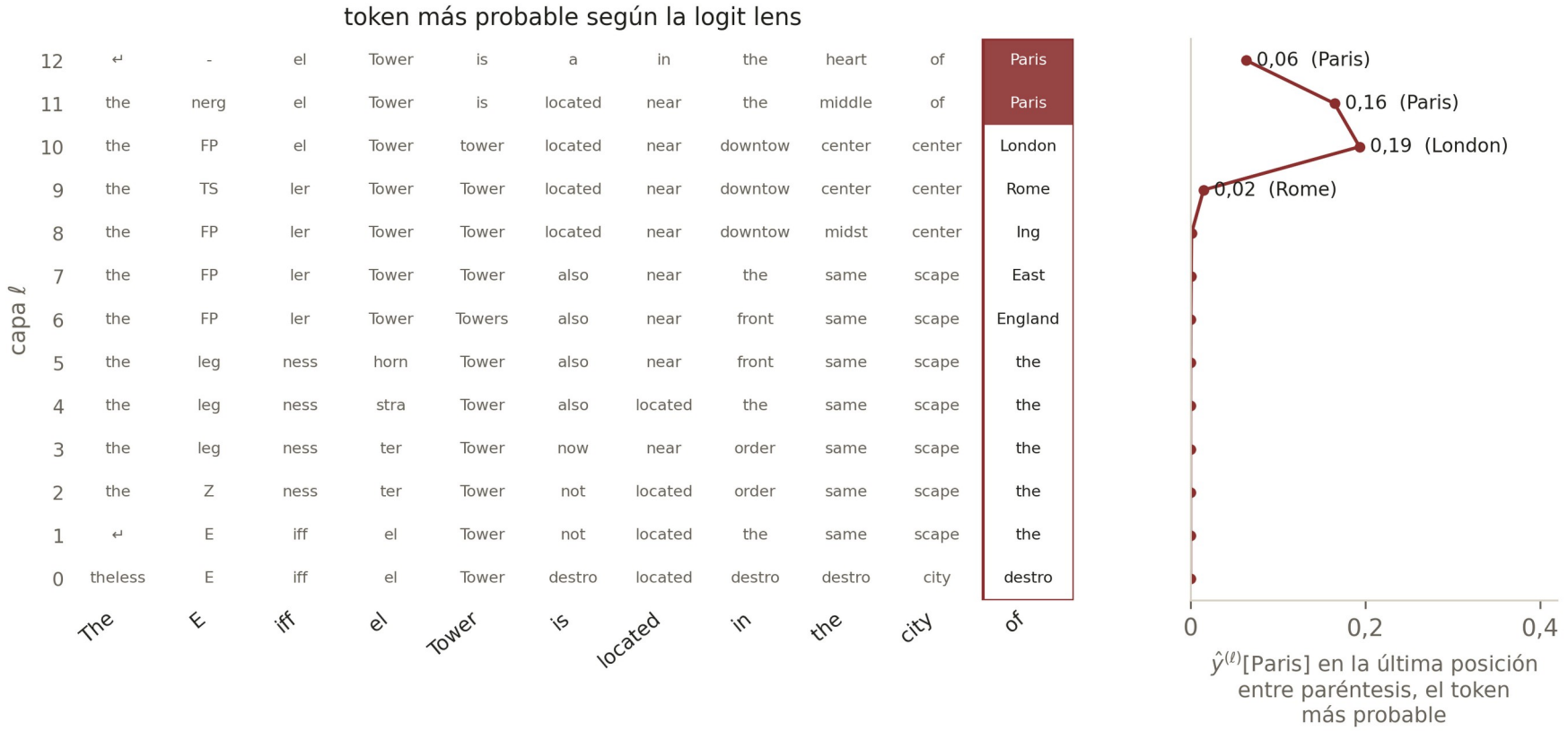


Las sombras del logit de Paris en GPT-2, una barra por subcapa. Las mayores vienen de las FFN de las capas 9 a 12.

nostalgebraist, 2020; Elhage et al., 2021. GPT-2 small, cálculo propio.

Logit lens en GPT-2

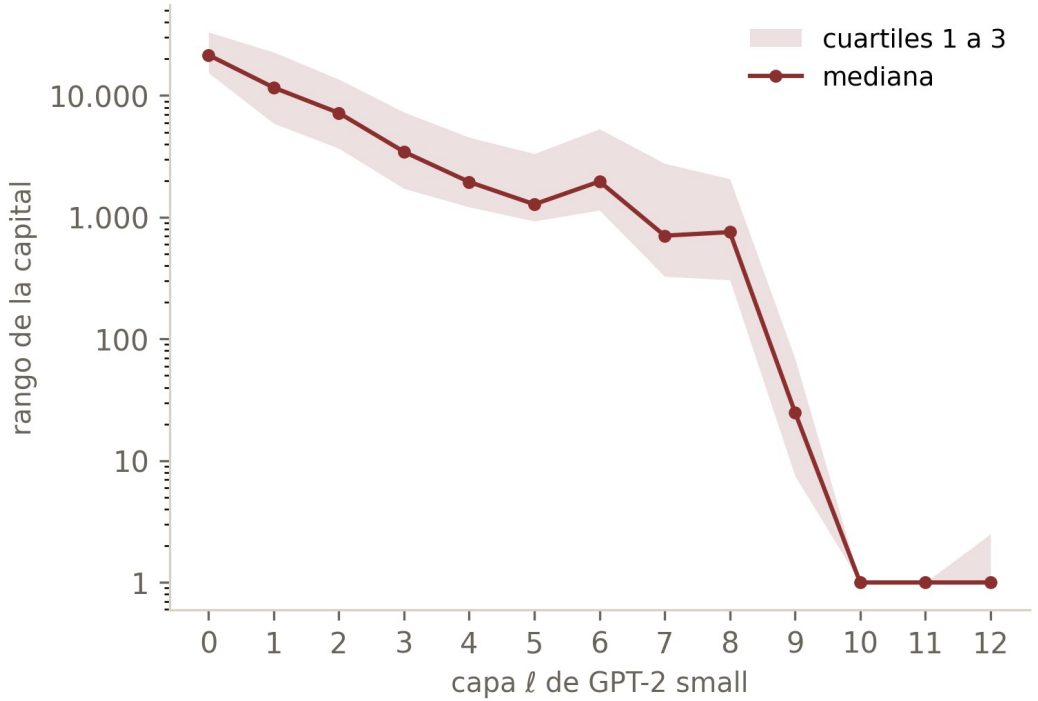
«The Eiffel Tower is located in the city of»: en la última posición, la capa 9 predice Rome, la 10 London y desde la 11 Paris. El modelo decide primero que viene una ciudad y después cuál.



GPT-2 small, 12 capas. Cálculo propio.

Logit lens sobre un conjunto de hechos

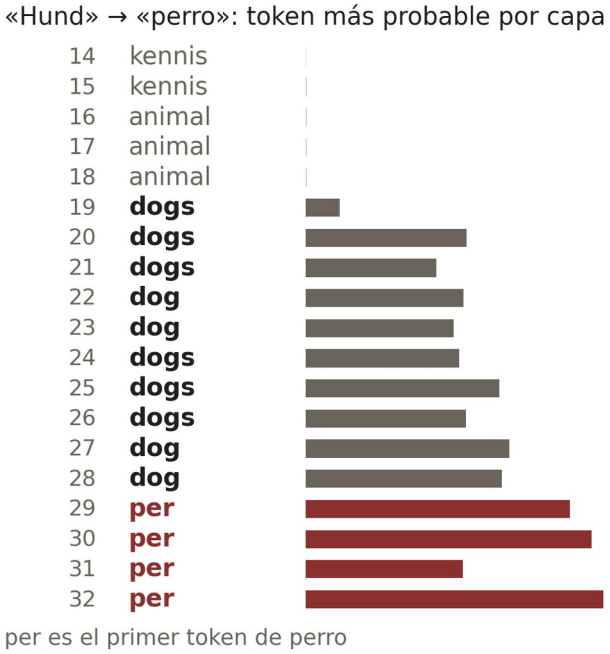
Con «The capital of X is the city of» y 23 países, la mediana del rango de la capital baja de 21.505 en el embedding a 1 desde la capa 10. La dinámica de la Torre Eiffel se repite.

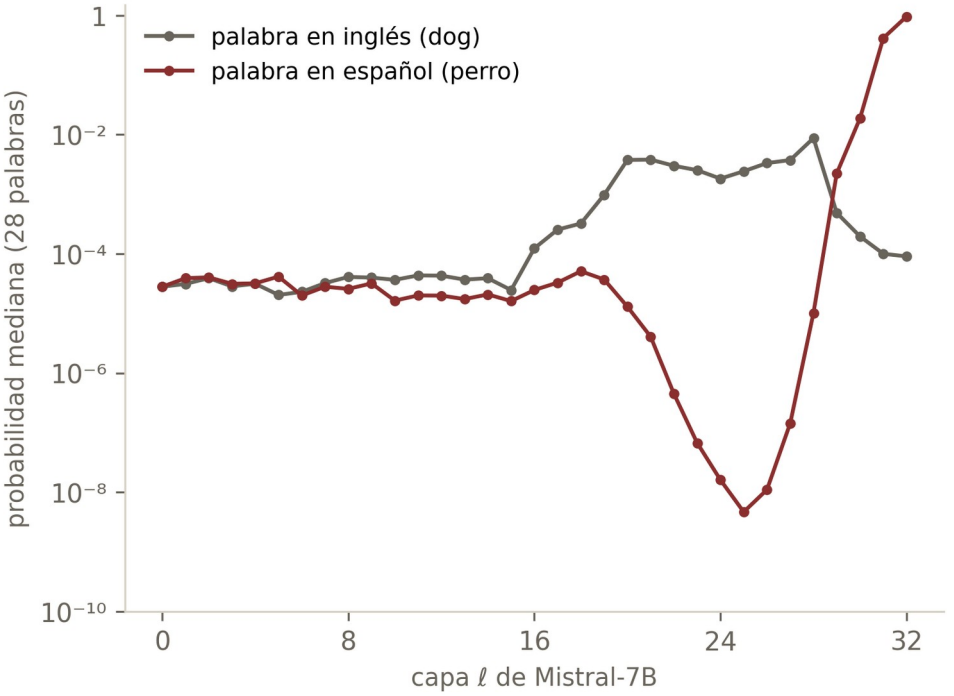


GPT-2 small, 23 capitales de un token. Cálculo propio.

Logit lens en traducción

Mistral-7B traduce del alemán al español. En las capas medias, la logit lens da más probabilidad a la palabra en inglés, que no aparece en el prompt, que a la palabra en español. El español se impone desde la capa 29.

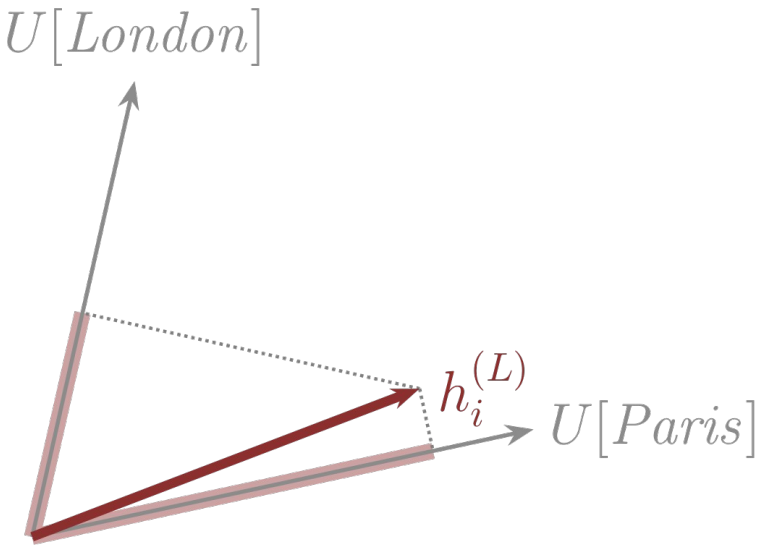




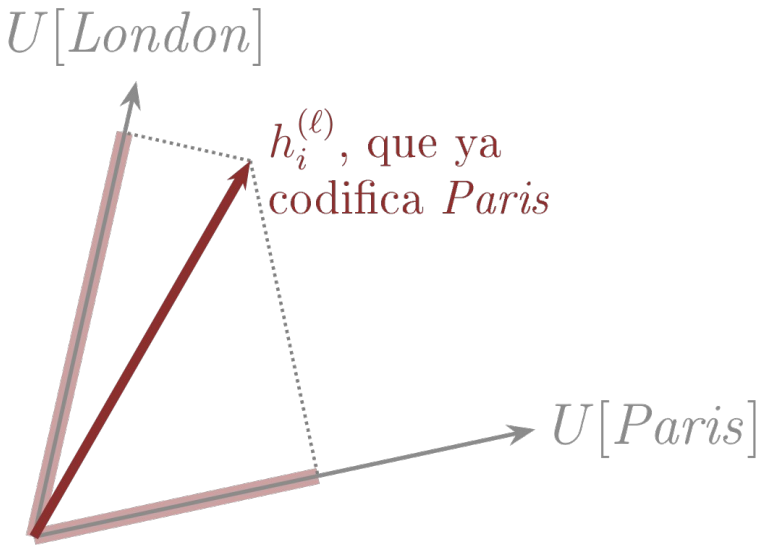
Mistral-7B, 28 palabras con un prompt de cuatro ejemplos. Cálculo propio, como en Wendler et al., ACL 2024 (Do Llamas Work in English?).

Por qué la logit lens puede fallar

U lee el residual por direcciones fijas, una por token. El entrenamiento solo obliga a la última capa a escribir en esas direcciones. Una capa intermedia puede guardar la misma información en otras.



capa final
la sombra mayor es *Paris*



capa intermedia
la sombra mayor es *London*

Belrose et al., 2023 (Eliciting Latent Predictions with the Tuned Lens).

Tuned lens

Si la capa ℓ escribe en otras direcciones, se aprende una transformación que las lleva a las direcciones que U lee. Hay un traductor afín por capa.

$$\hat{y}_i^{(\ell)} = \mathrm{softmax}\big(U\,\mathrm{LN}_f(\boldsymbol{A_\ell}\,h_i^{(\ell)} + \boldsymbol{b_\ell})\big) \qquad A_\ell \in \mathbb{R}^{d\times d},\ b_\ell \in \mathbb{R}^d\text{: un traductor por capa}$$

$$\min_{A_\ell, b_\ell} \quad \mathbb{E}_x\Big[\mathrm{KL}\big(\hat{y}_i \,\big\|\, \hat{y}_i^{(\ell)}\big)\Big] \qquad \text{imitar la predicción final; el modelo queda fijo}$$

$$A_\ell = I, \quad b_\ell = 0 \ \text{ al inicio} \qquad \text{parte desde la logit lens}$$

Qué se entrena

Solo A_ℓ y b_ℓ, con el modelo congelado, minimizando la KL hacia la predicción final.

Parte desde la logit lens

Con $A_\ell = I$ y $b_\ell = 0$ es la logit lens. El entrenamiento aprende solo la corrección.

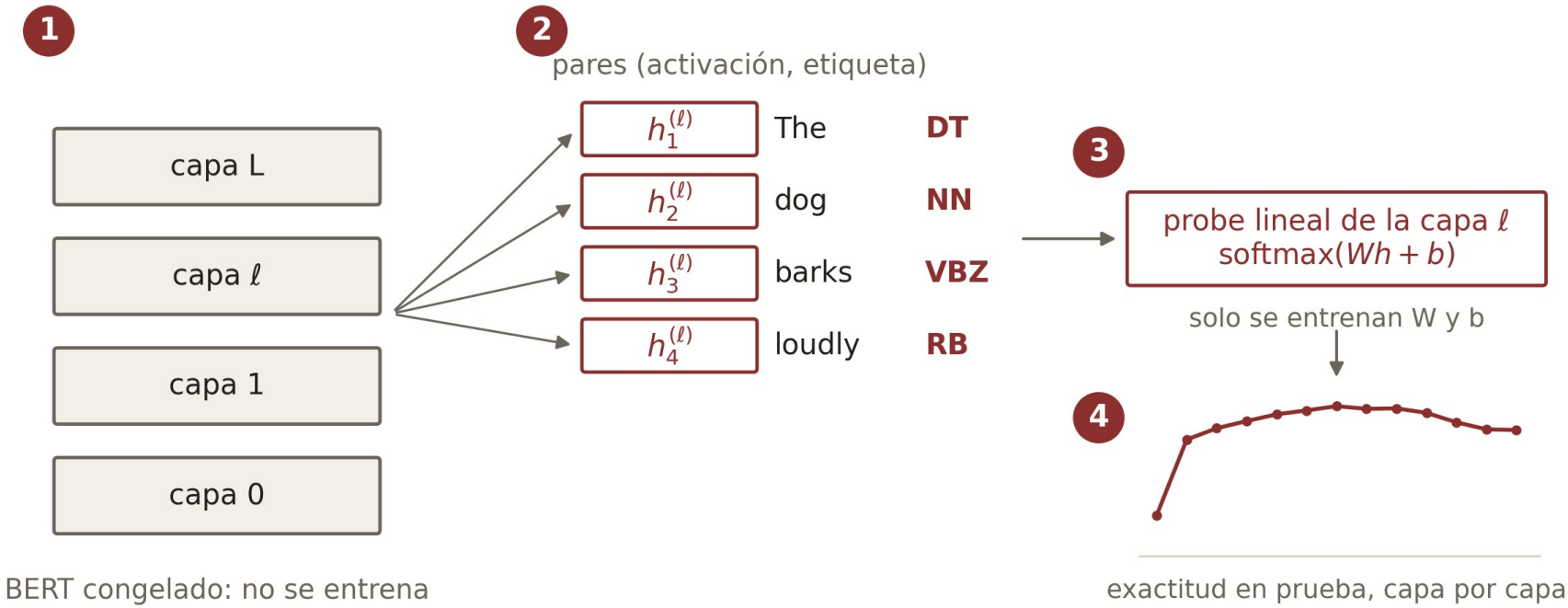
Cautela

Una probabilidad baja en la logit lens no implica que la información falte en esa capa, porque puede estar en otras direcciones.

Belrose et al., 2023 (Eliciting Latent Predictions with the Tuned Lens).

Probing: el mecanismo

El modelo se congela. Por cada palabra anotada se guarda su activación en la capa ℓ junto con su etiqueta, y con esos pares se entrena un clasificador lineal por capa.



BERT congelado: no se entrena

exactitud en prueba, capa por capa

Alain y Bengio, 2016; Belinkov, Computational Linguistics 2022. La curva es la del probe de POS en BERT-base, cálculo propio.

Probing: la definición

Un probe es una regresión logística sobre activaciones congeladas. Si su exactitud en prueba es alta, la propiedad se puede decodificar linealmente de esa capa.

$$\mathcal{D}^{(\ell)} = \{(h_n^{(\ell)}, y_n)\}_{n=1}^{N}, \quad h_n^{(\ell)} \in \mathbb{R}^d,\ y_n \in \{1, \dots, K\}$$

una activación por palabra, con su etiqueta

$$p\big(y \mid h_n^{(\ell)}\big) = \mathrm{softmax}\big(\boldsymbol{W}\, h_n^{(\ell)} + \boldsymbol{b}\big)$$

$W \in \mathbb{R}^{K \times d}$, $b \in \mathbb{R}^K$: lo único que se entrena

$$\mathcal{L}(W, b) = -\frac{1}{N} \sum_{n=1}^{N} \log p\big(y_n \mid h_n^{(\ell)}\big)$$

entropía cruzada; el modelo no recibe gradiente

$$\mathrm{acc}^{(\ell)} = \frac{1}{N'} \sum_{n \in \text{prueba}} \mathbf{1}\Big[\arg\max_k\, p\big(k \mid h_n^{(\ell)}\big) = y_n\Big]$$

en N' palabras que no se usaron para entrenar

Primera pieza

Con subwords, la activación de una palabra es la de su primera pieza.

Separación

Las palabras de prueba no se usan para entrenar el probe. Lo ideal es que vengan de otras oraciones.

Qué se concluye

Una exactitud alta muestra que la información está en $h^{(\ell)}$ y es accesible con una frontera lineal.

Alain y Bengio, 2016; Belinkov, Computational Linguistics 2022.

Cómo se lee la exactitud de un probe

La exactitud sola no basta. Se compara contra la del embedding y contra la clase mayoritaria, y el probe se mantiene lineal para que mida la capa y no al clasificador.

$$\Delta \mathrm{acc}^{(\ell)} = \mathrm{acc}^{(\ell)} - \mathrm{acc}^{(0)} \qquad \text{lo que la capa } \ell \text{ agrega sobre el embedding}$$

$$\mathrm{acc}_{\mathrm{may}} = \max_k \frac{|\{n : y_n = k\}|}{N} \qquad \text{predecir siempre la etiqueta más frecuente}$$

Contra el embedding

En POS, el probe sobre BERT-base acierta 0,872 ya en la capa 0, porque muchas palabras tienen una sola categoría.

Lineal

Un MLP grande aprende la tarea desde casi cualquier representación. Uno lineal mide lo que la capa ya trae.

Memorización

Aun lineal, un probe puede acertar memorizando la etiqueta de cada palabra. Eso lo mide la tarea de control.

Belinkov, Computational Linguistics 2022; Hewitt y Liang, EMNLP 2019.

Tareas de control

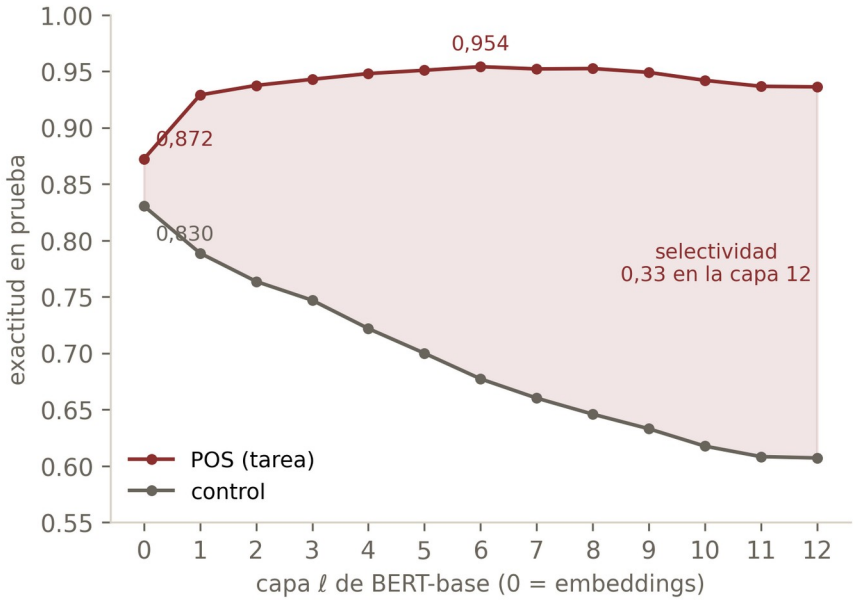
Un probe puede acertar memorizando palabras. La tarea de control asigna a cada palabra una etiqueta al azar; la selectividad es cuánto mejor le va al probe en la tarea real.

$$c(w) \sim \mathrm{Cat}(p_1, \ldots, p_K), \text{ fija para cada tipo de palabra } w$$

etiqueta al azar, con la distribución de la tarea

$$\mathrm{sel}^{(\ell)} = \mathrm{acc}^{(\ell)}_{\text{tarea}} - \mathbf{acc}^{(\ell)}_{\mathbf{control}}$$

lo que el probe no puede explicar memorizando



El experimento

Probe lineal de POS en las 13 capas de BERT-base: 42.471 palabras del Penn Treebank, 45 etiquetas.

La tarea

Llega a 0,954 en la capa 6. En el embedding ya es 0,872.

El control

Baja de 0,830 a 0,607, porque en las capas altas el probe ya no puede memorizar la palabra. La selectividad sube de 0,04 a 0,33.

Hewitt y Liang, EMNLP 2019 (Designing and Interpreting Probes with Control Tasks). Cálculo propio.

BERT y el pipeline clásico

Con probes sobre todas las capas de BERT, las tareas sintácticas se resuelven en capas bajas y las semánticas en capas más altas, en el orden del pipeline clásico de NLP.

$$h_{i,\tau} = \gamma_\tau \sum_{\ell=0}^{L} s_\tau^{(\ell)}\, h_i^{(\ell)}, \qquad s_\tau = \mathrm{softmax}(\alpha_\tau)$$

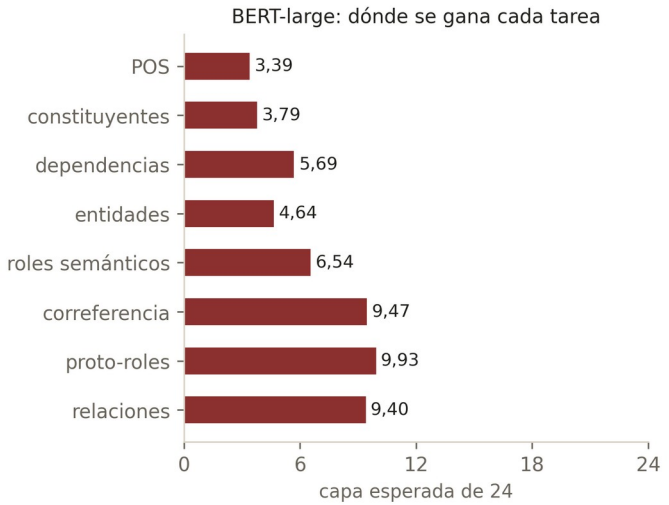
α_τ, γ_τ aprendidos: el peso de cada capa y una escala

$$\Delta_\tau^{(\ell)} = \mathrm{F1}\big(P_\tau^{(\ell)}\big) - \mathrm{F1}\big(P_\tau^{(\ell-1)}\big)$$

$P_\tau^{(\ell)}$: probe que solo ve las capas 0 a ℓ

$$\bar{E}_\Delta[\ell] = \frac{\sum_{\ell=1}^{L} \ell\, \Delta_\tau^{(\ell)}}{\sum_{\ell=1}^{L} \Delta_\tau^{(\ell)}}$$

capa esperada: dónde se gana el desempeño



Mezcla escalar

El probe de la tarea τ combina las capas con pesos s_τ aprendidos.

Capa esperada

Se agregan capas de a una y se mide cuánto sube el F1. El promedio de ℓ ponderado por esa ganancia es la capa donde se resuelve la tarea.

Resultado

POS en la capa 3,4 y dependencias en 5,7; correferencia en 9,5, de 24.

Tenney, Das y Pavlick, ACL 2019 (BERT Rediscovers the Classical NLP Pipeline). BERT-large.

Structural probe

Un probe no tiene que ser un clasificador. Este busca una proyección lineal en la que la distancia entre dos palabras sea su distancia en el árbol de dependencias.

$$d_B(h_i, h_j)^2 = \big(B(h_i - h_j)\big)^\top \big(B(h_i - h_j)\big) \qquad B \in \mathbb{R}^{k \times d}\text{: una proyección lineal}$$

$$\min_B \quad \sum_s \frac{1}{|s|^2} \sum_{i,j} \Big| d_T(w_i, w_j) - d_B(h_i, h_j)^2 \Big| \qquad d_T\text{: distancia en el árbol de dependencias de } s\text{; } |s|\text{: sus palabras}$$

$$\|h_i\|_B^2 = (Bh_i)^\top (Bh_i) \;\approx\; \text{profundidad de } w_i \qquad \text{la norma estima la profundidad en el árbol}$$

Qué se aprende

Solo la matriz B. Con ella se reconstruye el árbol completo: el árbol de expansión mínima de las distancias predichas.

Resultado

Con BERT-large, capa 15, se recuperan el 82,5 % de las aristas (UUAS), contra 59,8 % de la mejor línea base sin contexto.

Rango

Una proyección a k = 64 o 128 dimensiones basta, así que la sintaxis ocupa un subespacio chico.

Hewitt y Manning, NAACL 2019 (A Structural Probe for Finding Syntax in Word Representations).

Lo que un probe no dice

Que una propiedad se pueda leer de una capa no significa que el modelo la use. Para saberlo hay que intervenir.

Probe

lee una capa

Muestra que una propiedad se puede decodificar de $h^{(\ell)}$.

No muestra que el modelo la use para predecir.

Un probe muy capaz puede aprender la tarea por su cuenta.

La evidencia es una correlación.

Intervención

cambia una capa

Modifica $h^{(\ell)}$ y observa la salida del modelo.

Si la salida cambia como se espera, el modelo usa esa información.

Requiere elegir qué cambiar y cómo medir el efecto.

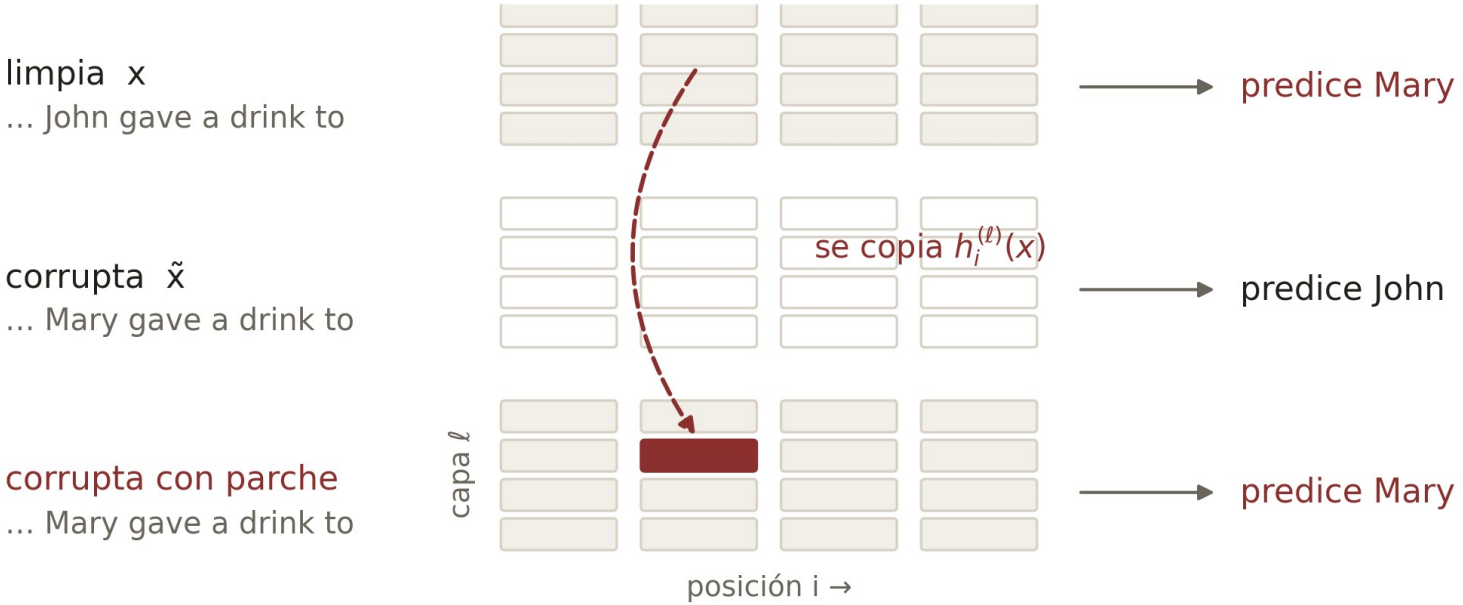
La evidencia es causal.

Fuente

Belinkov, Computational Linguistics 2022 (Probing Classifiers: Promises, Shortcomings, and Advances).

Activation patching: el mecanismo

Se corre el modelo con dos entradas que difieren en un solo token y dan respuestas distintas. Después se repite la corrupta copiando un estado de la limpia: si vuelve la respuesta limpia, ese estado transporta la información.



Vig et al., 2020; Meng et al., NeurIPS 2022; Wang et al., ICLR 2023; Heimersheim y Nanda, 2024.

Activation patching: la definición

El efecto de un componente es la fracción de la diferencia de logits entre la corrida limpia y la corrupta que se recupera al copiarlo.

$$\mathrm{LD}(x) = o_T(x)[r] - o_T(x)[\tilde{r}]$$

o_T: logits en la última posición; r,$\tilde{r}$: respuestas de x y de $\tilde{x}$

$$\mathrm{LD}(x) > 0, \qquad \mathrm{LD}(\tilde{x}) < 0$$

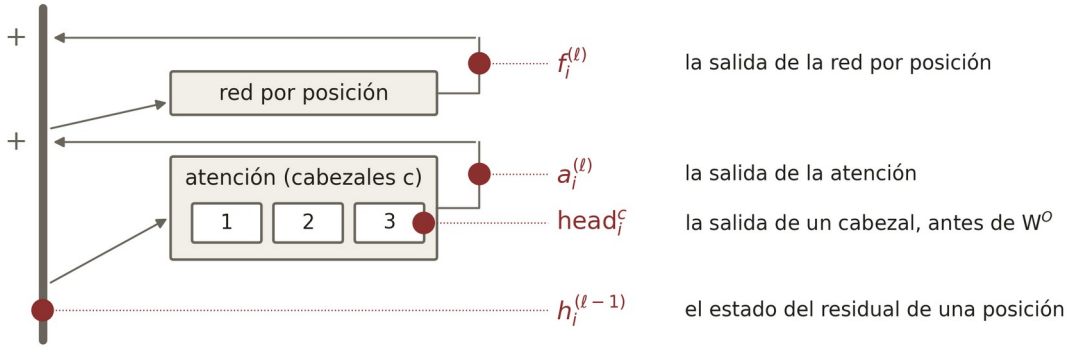
corrida limpia x y corrupta $\tilde{x}$, que difieren en un token

$$\mathrm{LD}_c = \mathrm{LD}\big(\tilde{x} \mid c \leftarrow \boldsymbol{c(x)}\big)$$

corrida corrupta con el componente c copiado de la limpia

$$\text{efecto}(c) = \frac{\mathrm{LD}_c - \mathrm{LD}(\tilde{x})}{\mathrm{LD}(x) - \mathrm{LD}(\tilde{x})}$$

0: el parche no cambia nada; 1: recupera toda la diferencia



Wang et al., ICLR 2023; Heimersheim y Nanda, 2024.

Componente

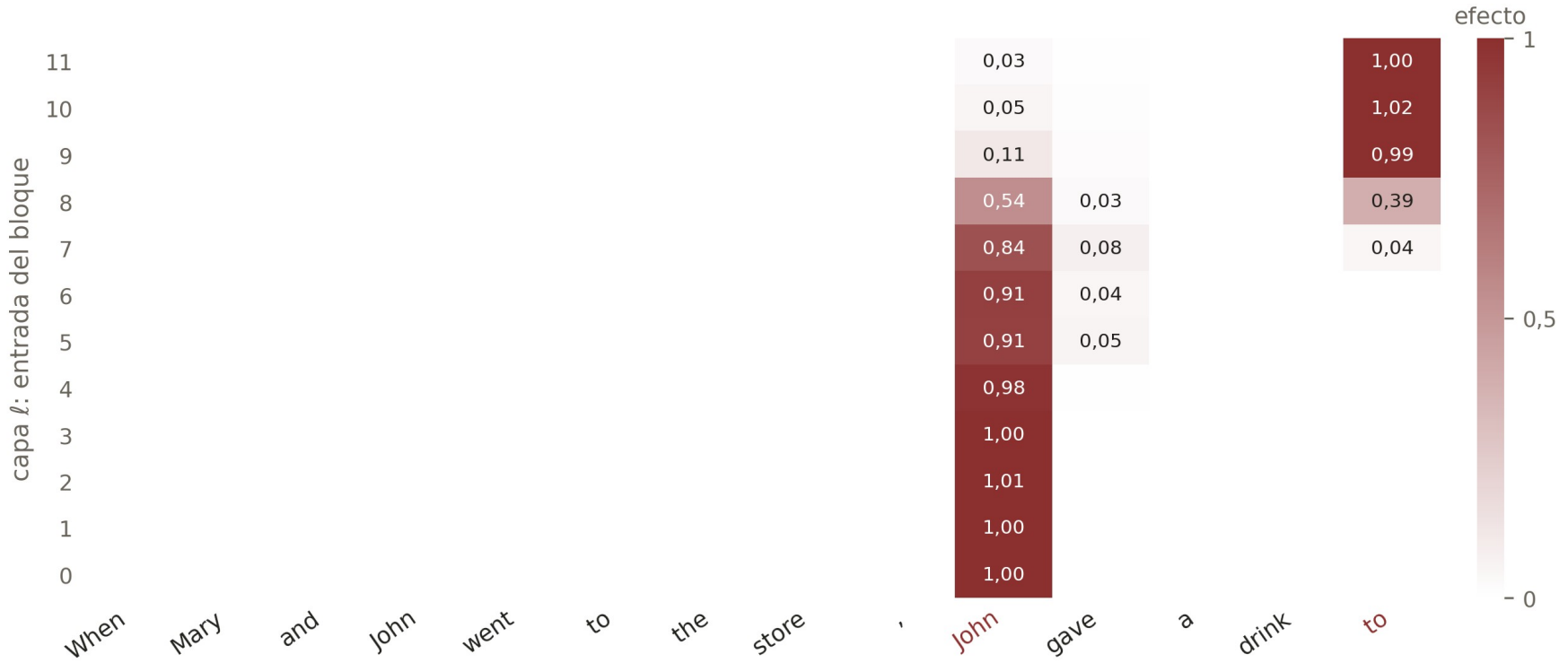
c puede ser el residual de una posición, la salida de una subcapa o la de un cabezal.

Costo

Un forward por componente. Un mapa por capa y posición en GPT-2 small, con una oración de 14 tokens, necesita 12 × 14 = 168 corridas.

Patching en IOI

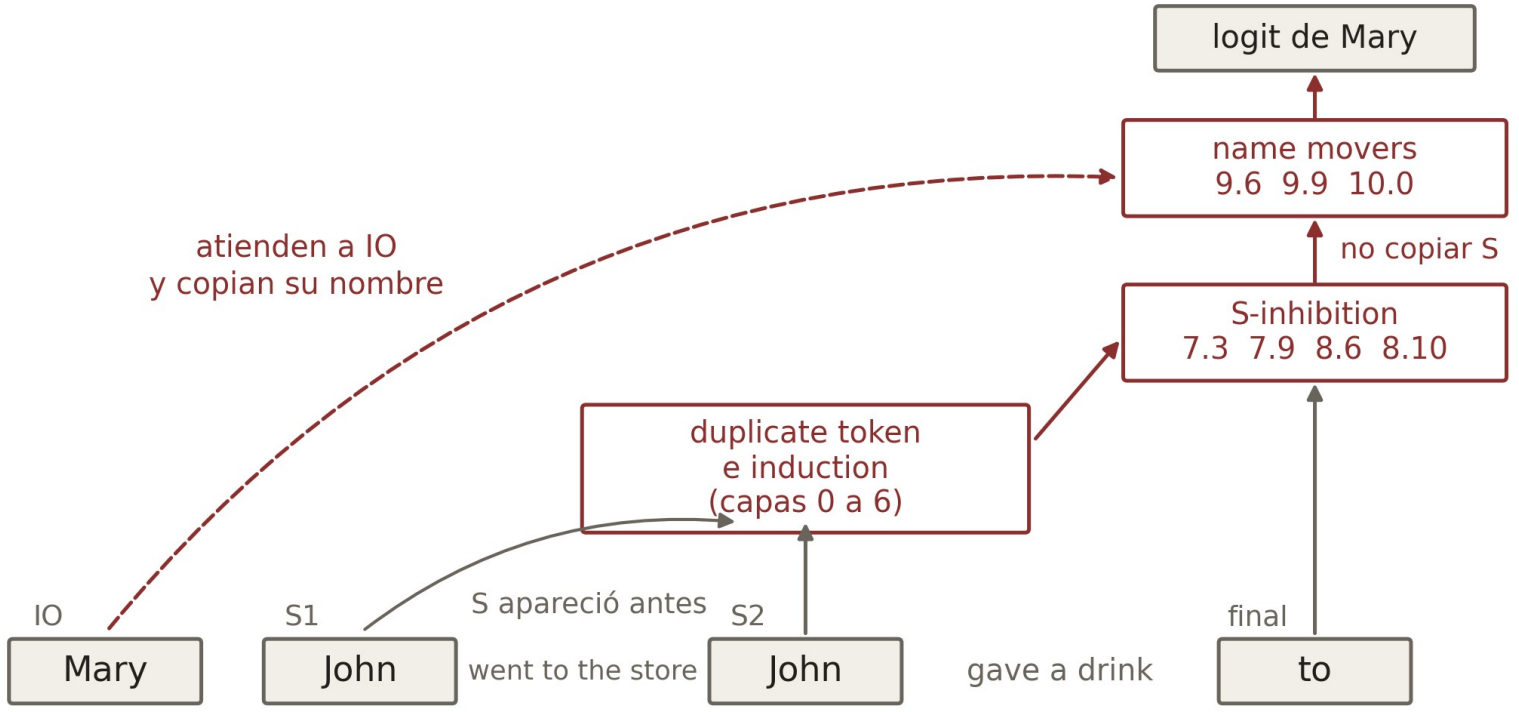
IOI (indirect object identification) es completar con el objeto indirecto, el nombre que no se repite. Hasta la capa 7 lo decisivo está en el segundo John, y desde la 9 en la última posición.



GPT-2 small. Entrada corrupta: el segundo John cambiado por Mary. Cálculo propio.

El circuito IOI

Patching cabezal por cabezal encuentra quién hace qué: unos detectan que John está repetido, otros inhiben copiarlo, y los name movers copian Mary a la salida.



Wang, Variengien, Conmy, Shlegeris y Steinhardt, ICLR 2023 (Interpretability in the Wild). 26 cabezales en 7 clases; simplificado.

Límites del patching

El patching da evidencia causal, pero cuesta muchas corridas y sus mapas hay que leerlos con cuidado.

Límite	Qué pasa	En los ejemplos
costo	un forward por componente; crece con capas, posiciones y cabezales	168 corridas para el residual de IOI
efectos repartidos	si varios componentes cumplen la misma función, parchar uno recupera poco	los name movers de respaldo de IOI
efectos negativos	un componente puede empujar en contra de la respuesta	los negative name movers de IOI
dónde, no cómo	dice en qué capa y posición la información influye en la respuesta, pero no qué cálculo hace el modelo con ella	el mapa del residual de IOI no muestra el circuito; hubo que parchar cabezal por cabezal
corrupción	el resultado depende de qué entrada corrupta se elija	ruido gaussiano y reemplazo de tokens dan mapas distintos

Wang et al., ICLR 2023; Zhang y Nanda, ICLR 2024; Heimersheim y Nanda, 2024.

Tres técnicas, una pregunta cada una

Las tres técnicas se complementan. La logit lens dice cuándo aparece la respuesta, el probing dónde está la información y el patching si el modelo la usa.

Técnica	Qué mide	Requiere	Evidencia	Ejemplo en la clase
logit lens	qué predice cada capa	un forward	lectura	Paris en GPT-2 desde la capa 11; inglés latente al traducir en Mistral-7B
probing	qué propiedad se lee de una capa	etiquetas y un probe por capa	correlación	POS en BERT-base, máximo en la capa 6; capas de Tenney
activation patching	qué componente transporta la información	par mínimo y un forward por componente	causal	IOI, del segundo John a la última posición entre las capas 7 y 9

nostalgebraist, 2020; Belinkov, Computational Linguistics 2022; Heimersheim y Nanda, 2024.